



TED UNIVERSITY

CMPE 492 – Senior Project II

Spring 2026

AKKE – Low Level Design Report Version 2.0

(SMART COMMAND AND CONTROL GLOVE)

Webpage: <https://ilter-akke.github.io/website/>

Name Surname Department ID

Abdullah ESİN (CMPE) 14476636120

Berk ÇAKMAK (CMPE) 10948064620

Ömer Efe DİKİCİ (CMPE) 63058185488

Şevval KURTULMUŞ (EEE) 52654371036

Supervisors: Ali BERKOL and Hüseyin Uğur YILDIZ

Jury Members: Hakkı Gökhan İLK and Mehmet Evren COŞKUN

Contents

1. Introduction and Scope.....	3
1.1 Purpose	3
1.2 System Overview.....	3
1.3 Scope – Items to Be Tested	3
1.4 Items NOT Tested	4
2. Testing Methodology.....	4
2.1 Unit Testing	4
2.2 Integration Testing.....	4
2.3 System Testing	4
2.4 Performance Testing.....	4
2.5 User Acceptance Testing (UAT)	4
3. Features to Be Tested	5
4. Test Environment	5
4.1 Hardware Components	5
4.2 Software & Firmware	6
4.3 Test Locations	6
5. Test Schedule.....	6
5.1 Sprint-Aligned Testing Timeline.....	6
5. Control Procedures	6
5.1 Test Execution Process.....	6
5.2 Defect Reporting	6
5.3 Change Control.....	7
5.4 Entry and Exit Criteria.....	7
6. Roles and Responsibilities.....	7
7. Risks and Mitigation.....	7
8. References	8

1. Introduction and Scope

1.1 Purpose

This Test Plan defines the scope, approach, resources, schedule, and responsibilities for all testing activities of the AKKE (Smart Command and Control Glove) project, developed as part of CMPE 492 – Senior Design Project II at TED University. AKKE is a wearable gesture-recognition system that enables silent, secure, real-time team communication for tactical, rescue, and noisy industrial environments. This document ensures the final prototype satisfies all functional and non-functional requirements before the final demonstration (D8, T31) and poster exhibition (T32) in May 2026.

1.2 System Overview

AKKE translates predefined hand gestures into wireless audio commands transmitted via ESP-NOW to team receivers. The current sprint status (April 2026):

- Sensor firmware (T21) — COMPLETED: 200 Hz C++ ADC driver with 0/4095 hardware fault detection.
- Gesture data collection (T23) — IN PROGRESS: streaming Timestamp + 5 Flex + 6 IMU over Serial.
- TinyML model training (T24) — PENDING: model to be trained on collected dataset.
- ESP-NOW integration (T26) — IN PROGRESS: digital trigger between glove and receiver.
- Audio module setup (T27) and battery/LED (T29) — PENDING.

Hardware: ESP32-WROOM-32 microcontroller; 5 flex sensors (GPIO 25, 33, 32, 35, 34 via ADC); MPU-6050 IMU (I2C: SDA GPIO 21, SCL GPIO 22 at 400 kHz); microSD storage; 3.7 V Li-ion battery (glove), 9 V battery (receiver).

Software: On-device TinyML gesture classifier (TensorFlow Lite Micro); I2S audio pipeline; ESP-NOW encrypted peer-to-peer link; C++ firmware (Arduino).

Targets: 5 gestures initially (expandable to 50+); 75–80% recognition accuracy; end-to-end latency < 100 ms with antenna optimisation; 25 beta gloves; mobile app v1.0.

1.3 Scope – Items to Be Tested

- FlexSensor driver (ADC, 200 Hz, fault detection — T21 completed)
- IMUSensor driver (I2C MPU-6050, 3-axis acc + gyro — T21 completed)
- Serial sensor data stream for dataset collection (T23 — In Progress)
- TinyML GestureClassifier with Decision Gate (T24 — Pending)
- ESP-NOW communication layer – ESPNowSender / ESPNowReceiver (T26 — In Progress)
- DFPlayer Mini audio playback via UART (T27 — Pending)
- Receiver battery management via voltage divider, RGB LED feedback (T29 — Pending)
- GloveController state machine (all 7 states and transitions)
- End-to-end pipeline latency, accuracy, and range (T28 — Pending)

1.4 Items NOT Tested

- Cloud connectivity or remote management (system is intentionally fully offline)
- Full 50+ gesture set (only 5 prototype gestures tested in this plan)
- Mobile app v1.0 (out of scope for current sprint cycle)
- Environmental durability (dust, moisture, extreme temperature) — noted as design intention; not validated this semester
- Security penetration of the ESP-NOW encrypted channel

2. Testing Methodology

The AKKE test strategy uses a bottom-up integration approach aligned with the project's sprint structure (Sprints 7–9). Unit tests of individual drivers are conducted first, followed by integration tests of subsystem interfaces, and culminating in full system and user-acceptance tests.

2.1 Unit Testing

Individual C++ classes (FlexSensor, IMUSensor, GestureClassifier, ESPNowSender, AudioEngine) are tested in isolation. Pre-recorded sensor vectors are used for ML tests where real-time hardware cannot be isolated. Timing assertions verify the 200 Hz sampling rate and < 50 ms inference constraints.

2.2 Integration Testing

Pairs and groups of subsystems are tested for correct data flow: sensor data → FeatureVector → TinyML inference → ESP-NOW → DFPlayer. Integration tests also cover the Accuracy Gate, calibration procedure, battery monitoring, and receiver LED feedback (T29).

2.3 System Testing

The full assembled prototype is tested end-to-end against all functional and non-functional requirements (T28). System tests cover sub-100 ms latency, $\geq 75\%$ gesture accuracy, state machine lifecycle, movement timeout, and multi-receiver simultaneous broadcast.

2.4 Performance Testing

Dedicated performance tests quantify: end-to-end latency (target < 100 ms per HowItWorksSection website specification), TinyML inference (< 50 ms), 200 Hz sampling stability, and 60-minute continuous operation. Results will be reported in the Final Report (T30).

2.5 User Acceptance Testing (UAT)

UAT sessions with volunteer participants assess ease of setup, gesture learnability within 10 minutes, and LED feedback comprehension without a manual — targeting the FAQ claim of an accessible, headless physical UI.

3. Features to Be Tested

Test ID	Feature / Subsystem	Category	Priority	Sprint Link
TC-U-001	Flex Sensor Raw ADC Read at 200 Hz	Unit Test	High	T24/T28
TC-U-002	Flex Sensor Calibration and Normalisation	Unit Test	High	T24/T28
TC-U-003	IMU Sensor Data Validity and Streaming (T21)	Unit Test	High	T24/T28
TC-U-004	Serial Sensor Data Stream Format (T23)	Unit Test	High	T24/T28
TC-U-005	TinyML GestureClassifier – Advance Gesture (T24)	Unit Test	High	T24/T28
TC-U-006	TinyML Classifier – Low Confidence Returns 0xFF	Unit Test	High	T24/T28
TC-U-007	TinyML Inference Timing Constraint	Unit Test	High	T24/T28
TC-I-001	Sensor → Feature Vector → Classifier Pipeline	Integration Test	High	T26-T29
TC-I-002	ESP-NOW Digital Trigger Transmission and ACK (T26)	Integration Test	High	T26-T29
TC-I-003	Audio Command Playback on Receiver (T27)	Integration Test	High	T26-T29
TC-I-004	Accuracy Gate – Low Confidence Blocks Transmission	Integration Test	High	T26-T29
TC-I-005	Receiver Battery Management and LED Feedback (T29)	Integration Test	Medium	T26-T29
TC-I-006	Glove Calibration Procedure Integration	Integration Test	Medium	T26-T29
TC-S-001	All 5 Gesture Recognition Accuracy $\geq 75\%$ (T28)	System Test	High	T28
TC-S-002	ESP-NOW Range and Reliability Test	System Test	High	T28
TC-S-003	GloveController State Machine Lifecycle	System Test	High	T28
TC-S-004	Movement Timeout – Return to Idle (3 Seconds)	System Test	Medium	T28
TC-P-001	Continuous Operation Stability – 60 Minutes	Performance Test	High	T28
TC-P-002	Sensor Sampling Rate Consistency – 200 Hz	Performance Test	Medium	T28
TC-A-001	First-Time User Setup and Calibration (< 3 min)	User Acceptance Test	High	T28
TC-A-002	Gesture Learnability – 5 Gestures in 10 Minutes	User Acceptance Test	Medium	T28
TC-A-003	LED Feedback State Comprehension	User Acceptance Test	Low	T28

4. Test Environment

4.1 Hardware Components

Component	Specification / Role
Glove MCU	ESP32-WROOM-32 — main processing, TinyML inference, ESP-NOW sender
Receiver MCU	ESP32-WROOM-32 — ESP-NOW receiver, DFPlayer control, LED/battery
Flex Sensors	5 × resistive flex, GPIO 25/33/32/35/34 via 12-bit ADC
IMU	MPU-6050 — 3-axis acc + 3-axis gyro, I2C 400 kHz, SDA GPIO21, SCL GPIO22
Audio Module	DFPlayer Mini — UART 9600 baud, microSD (5 WAV command files)
Power (Glove)	9 V battery with voltage divider for level detection
Power (Receiver)	9 V battery with voltage divider for level detection
RGB LED	Common-anode RGB LED, PWM-driven GPIO
Calibration Button	Physical push-button on GPIO interrupt pin
Measurement Tools	Logic analyser, Arduino Serial Monitor, stopwatch

4.2 Software & Firmware

- Language: C++ (Arduino framework)
- ML: TensorFlow Lite Micro — model trained in Python (Edge Impulse / TF2), converted to .tflite C array
- Repository: <https://github.com/ILTER-AKKE/website> (website); firmware repo TBD
- Serial logging: 115200 baud; CSV format: Timestamp, Flex1–5, Acc_X/Y/Z, Gyr_X/Y/Z (T23)

4.3 Test Locations

- Unit & Integration Tests: TED University Electric Electronic Engineering lab (Sprints 7–8)
- RF Range Tests: Open outdoor area, TED University campus
- UAT sessions: Controlled indoor room with volunteer participants (Sprint 9)

5. Test Schedule

5.1 Sprint-Aligned Testing Timeline

Sprint	Phase	Test Cases	Date Range	Backlog	Status
Sprint 7	Unit Testing (Sensors, ML)	TC-U-001 to TC-U-007	10–15 Apr 2026	T23, T24	Completed
Sprint 8	Integration Testing	TC-I-001 to TC-I-006	16 Apr – 5 May 2026	T26–T29	In Progress
Sprint 8	System & Perf. Testing	TC-S-001 to TC-S-004, TC-P-001, TC-P-002	1–5 May 2026	T28	Pending
Sprint 9	User Acceptance Testing	TC-A-001 to TC-A-003	6–14 May 2026	T31	Pending
Sprint 9	Final Report & Review	Compile results → D7 Final Report	15 May 2026	T30	Pending

6. Control Procedures

6.1 Test Execution Process

- All tests are executed against the firmware version tagged in the GitHub repository at the time of testing.
- Each test case is executed by the assigned team member and observed by at least one other member.
- Results (Pass / Fail / Blocked / N/A) are recorded in the test result log immediately after execution and linked to the corresponding backlog task ID.

6.2 Defect Reporting

- Any test failure is logged as a GitHub Issue with labels: test-failure, severity (Critical / High / Medium / Low), and the Test ID.
- Critical and High severity defects block the next test phase and must be resolved before T28 (End-to-End System Testing, 5 May 2026).
- Medium and Low defects are tracked for resolution before T30 (Final Report, 15 May 2026).

6.3 Change Control

- Any firmware change after the start of a test phase requires re-execution of affected test cases.
- The test plan version is updated whenever scope changes, new sprint tasks are added, or supervisory feedback is received (as occurred in T22 — LLD revision).

6.4 Entry and Exit Criteria

Entry Criteria:

- T18 (Glove Assembly & Soldering) substantially complete — hardware available for testing.
- T21 (Sensor Data Acquisition firmware) deployed and verified on physical hardware.
- T24 (TinyML Model) compiled and flashed for ML-dependent tests.

Exit Criteria:

- All High-priority test cases executed and passed.
- No Critical or High severity defects remain open.
- T28 End-to-End System Testing completed; results documented for T30 Final Report.

7. Roles and Responsibilities

Team Member	Role / GitHub	Primary Test Responsibilities	Secondary Responsibilities
Abdullah ESİN github.com/AbdullahESIN	CMPE	TC-U-001 (ADC/flex), TC-U-002, TC-U-004 (serial stream T23), TC-P-002 (200 Hz), TC-S-004, TC-I-006, TC-A-003	Test log maintenance; Sprint backlog updates (T20); dataset labelling for T24
Berk ÇAKMAK github.com/CakmakBerk	CMPE	TC-U-005 to TC-U-007 (TinyML, inference timing), TC-I-001 (pipeline), TC-P-001 (stability), TC-S-001 (accuracy)	ML model training (T24); performance analysis; inference optimisation
Ömer Efe DİKİCİ github.com/Aosll	CMPE	TC-I-004 (Accuracy Gate), TC-S-002 (ESP-NOW range), TC-S-003 (state machine), TC-A-001	UAT coordination; system integration; Serial protocol design
Şevval KURTULMUŞ github.com/SevvalKurtulmus	EEE	TC-U-003 (IMU), TC-I-002 (ESP-NOW T26), TC-I-003 (audio T27), TC-I-005 (battery T29)	Hardware assembly (T18); RF chain design; voltage divider circuit (T29)

8. Risks and Mitigation

#	Risk	Severity	Impact	Mitigation
1	TinyML model (T24) accuracy < 75% — dataset from T23 may be insufficient	High	TC-S-002 fails; overall project goal not met	Collect 200+ samples per gesture; augment with synthetic noise; use Edge Impulse auto-tuning
2	Broken flex sensors (T18 In Progress) delay hardware availability	High	Unit and integration tests blocked	Procure spare sensors; solder and test incrementally; run software-only simulation in parallel
3	ESP-NOW range insufficient at 10 m (T26 In Progress)	High	TC-I-002 and TC-S-004 fail; command delivery unreliable	Test with external antenna; adjust TX power; evaluate channel 1 vs 6 vs 11
4	9V receiver battery voltage drop during testing (T29 Pending)	Medium	TC-I-006 and TC-S-001 affected; erratic sensor/audio behavior	Complete T29 voltage divider circuit before T28
5	Gesture dataset class imbalance across T23 participants	Medium	ML model biased toward certain gestures; accuracy below 75%	Ensure equal sample counts per gesture per user; validate on held-out user
6	Insufficient UAT volunteers for T31 demo	Low	UAT results not statistically meaningful	Recruit 3+ volunteers from CMPE/EEE cohort by end of Sprint 8

9. References

- [1] A. ESİN, B. ÇAKMAK, Ö. E. DİKİCİ, Ş. KURTULMUŞ, "AKKE – Project Proposal", TED University, CMPE 491, Fall 2025.
- [2] A. ESİN et al., "AKKE – Project Specifications Report", TED University, CMPE 491, Fall 2025.
- [3] A. ESİN et al., "AKKE – Analysis Report", TED University, CMPE 491, Fall 2025.
- [4] A. ESİN et al., "AKKE – High Level Design Report", TED University, CMPE 491, Fall 2025.
- [5] A. ESİN et al., "AKKE – Low Level Design Report (v1 & v2)", TED University, CMPE 492, Spring 2026. (T19, T22)
- [6] ILTER-AKKE GitHub Repository: <https://github.com/ILTER-AKKE/website> — project backlog (TODO.csv), website source code.
- [7] Espressif Systems, "ESP32-WROOM-32 Datasheet & ESP-NOW Protocol Guide", 2024.
- [8] InvenSense, "MPU-6050 Six-Axis IMU Datasheet", 2013.
- [9] DFRobot, "DFPlayer Mini MP3 Player for Arduino (DFR0299)", DFRobot Wiki.
- [10] TensorFlow, "TensorFlow Lite for Microcontrollers", <https://www.tensorflow.org/lite/microcontrollers>
- [11] B. Bruegge and A. H. Dutoit, Object-Oriented Software Engineering: Using UML, Patterns, and Java, 2nd ed. Prentice Hall, 2004.
- [12] IEEE Std 829-2008, "Standard for Software and System Test Documentation", IEEE, 2008.